

		Цветовой код	
		Сделано	Частично/Иначе
		Отказ	
Задача	План	Выполнено	GIT
📅 Фаза 1: Рефакторинг и стабилизация движка (1-2 недели)	09.05.2026		
Цель: устранить технический долг, улучшить архитектуру, подготовить движок к расширению. Задачи:			
1. Удалить дублирующий код:			
Удалить старый GameObject и связанные с ним файлы из игры.	02.05.2026	02.05.2026	
Перенести актуальные функции для работы с текстом (GetTextOrigin, DrawTextList) в движок	02.05.2026	02.05.2026	
Временное отключение неиспользуемых файлов Menu, GameState, их заголовочных и связанных с ними	02.05.2026	02.05.2026	
Полностью перевести игру на использование XYZEngine .			
2. Улучшить управление памятью:			
Заменить сырые указатели на std::unique_ptr в SpriteRendererComponent	02.05.2026	02.05.2026	
Заменить сырые указатели на std::unique_ptr в AudioComponent	02.05.2026	02.05.2026	
Заменить сырые указатели на std::unique_ptr в CameraComponent	02.05.2026	02.05.2026	
Заменить сырые указатели на std::unique_ptr в ResourceSystem	02.05.2026		
Заменить сырые указатели на std::unique_ptr / std::shared_ptr в GameWorld и при хранении компонентов			
В ColliderComponent хранить слабые ссылки на подписчиков или использовать std::function с проверкой валидности объекта.			
3. Рефакторинг рендеринга:			
SpriteRendererComponent переименовать в SpriteComponent			
4. Улучшить систему коллизий:			
Добавить поддержку коллизий по сетке (grid) для оптимизации при большом количестве статических объектов (стены).	03.05.2026		
Реализовать BoxColliderComponent отдельно от SpriteColliderComponent, чтобы коллизия не зависела от спрайта.	03.05.2026		
5. Сделать простой дизайн-документ (Design Doc)	09.05.2026		
Минимальная структура:			
– Общее описание игры:			
• Жанр, цель игры, кто главный герой, кто враги, что нужно делать.		03.05.2026	
– Список механик и фич:			
• Что уже есть в игре (движение, враги, уровни, интерфейс и т.д.).		03.05.2026	
• Что добавляем самостоятельно (в рамках выбранного направления).		03.05.2026	
– Оценка фич по сложности (примерная, в сторипointах)			
• Какие задачи и подзадачи легче, какие сложнее, и сколько сторипointов им назначить.		03.05.2026	
– Таймлайн разработки по спринтам (примерный):			
• Написать, когда планировалось и когда реально сделана каждая фича		03.05.2026	
Можно оформить в виде простого списка по датам, с учетом того, что один спринт = одна неделя.			
– Работа с Git:			
• Какие ветки создавались			
• Какие Merge Requests делались, зачем, и что в них было.			
📅 Фаза 2: Базовый геймплей рогалика (2-3 недели)	16.05.2026		
Цель: реализовать минимально играбельную версию с процедурной генерацией, стрельбой и простым врагом. Задачи:			
1. Улучшение ИИ врагов:	04.05.2026		
Создать класс врага - Enemy, отнаследованный от нашего персонажа		04.05.2026	
Создать класс, который будет отвечать за спавн врагов - EnemySpawner.		04.05.2026	
Использовать этот класс для спавна врагов после завершения генерации карты и лабиринта.		04.05.2026	
Настроить класс спавнера динамически, чтобы он умел принимать параметры количества врагов, типа врагов и так далее.		04.05.2026	
Создать компонент AIBehaviorComponent с состояниями (Idle, Patrol, Chase, Attack).		04.05.2026	
Для начала реализовать ChaseAndAttack — преследование игрока в пределах видимости и атака при касании.			
Сформировать документацию для GDD по системе AI.			
2. Система стрельбы:	05.05.2026		
Компонент WeaponComponent, который создаёт Projectile (пуля) при нажатии клавиши мыши/пробела.		05.05.2026	
Пуля — отдельный GameObject с коллизией, своим спрайтом, движением по направлению и нанесением урона при столкновении		06.05.2026	
Дополнить AIBehaviorComponent логикой смерти врага		06.05.2026	
Исключить InputComponent и заменить его на синглтон InputManager		07.05.2026	
Скорректировать ColliderComponent и PhysicsSystem::Update() для обработки коллизии между динамическим объектами			
Сформировать документацию для GDD по боевой системе.			
3. Пользовательский интерфейс (UI) в игре:	07.05.2026		
– Придумать и реализовать полноценную систему интерфейса – меню паузы, экран победы/поражения, подсказки на экране, динамические элементы, инвентарь, всплывающие сообщения и прочее.			
– Реализовать отдельный класс для отрисовки UI-элементов, может быть widget, по аналогии с классами Unreal Engine		06.05.2026	
Интегрировать UI в движок, чтобы он рисовался поверх игрового мира (например, через CanvasComponent).			
• Создать UIManager или аналог, который будет активировать/скрывать нужные элементы по ситуации.		06.05.2026	
– Реализовать HUD (интерфейс), который показывает здоровье и выносливость (стамину) персонажа, иконку оружия, количество патронов		07.05.2026	branch Phase2.3-HUD
• Это может быть полоска или иконки, которые обновляются в реальном времени при получении урона или использовании выносливости.		07.05.2026	
• Для HUD удобно использовать "подписку" на события:			
o Получил урон → обновил здоровье.			
o Использовал выносливость → обновил стамину.			
• Разделить UI на экраны:			
o HUD		07.05.2026	
o Меню паузы		08.05.2026	branch Phase2.3-PauseMenu
o Экран инвентаря		08.05.2026	
o Реализовать синглтон Game для управления состояния игры		11.05.2026	branch Phase2.4-InventorySystem-Bugfixing-Game
o Главное меню		11.05.2026	
o Экран победы/поражения		14.05.2026	
o Всплывающие сообщения / подсказки		09.05.2026	
• Не забыть про блокировку управления, когда активны экраны (например, в меню паузы).			
Сформировать документацию для GDD по интерфейсу.			
4. Инвентарь (базовый):	09.05.2026		
– Сделать инвентарь, в который можно добавлять и использовать предметы.		08.05.2026	
• Например: аптечки, ключи, оружие. Игрок может собирать, выбирать и использовать их в бою или для прохождения.			
• Добавить кнопку (например, I), чтобы открыть/закрыть окно инвентаря.		08.05.2026	
– Класс Inventory как компонент игрока, хранящий список предметов (Item).		08.05.2026	
• Структура предмета Inventory — просто std::vector<Item>. Использовать — вызвать onUse().		08.05.2026	
Простые предметы: аптечка, патроны, ключ-карта.			
• Для взаимодействия с предметами сделать:			
o выбор в UI		08.05.2026	branch Phase2.4-InventorySystem-Interaction
o кнопку “использовать”		08.05.2026	
• Примеры эффектов			

о Аптечка: player->Heal(20); о Ключ: door->Unlock(); о Оружие: player->EquipWeapon(sword); ● Использовать событийную систему: "предмет применён" → "показать вспышку / звук / лог". Подбор предметов при касании (триггер-коллизия) - реализовано через InteractionComponent ● Отрисовку сделать циклом: for (auto& item : inventoryItems). — Префабы — Рефакторинг системы инвентаря и взаимодействия Сформировать документацию для GDD по инвентарю и взаимодействию		08.05.2026	
		12.05.2026	
		09.05.2026	
		09.05.2026	
		08.05.2026	
5. Процедурная генерация уровней: — Добавить новые префабы (дверь, ящик, компьютер, выход) и их обработчики 5.1 Создать класс MazeGenerator , алгоритм генерации лабиринта, использующий алгоритм Depth-First Search (DFS - поиск в глубину) . Он работает с абстрактной сеткой ячеек и возвращает карту тайлов (стены, полы, двери) ● Решить проблему с обработкой алгоритмом внешних стен карты 5.2 Создать LevelBuilder – строитель уровня. Получает карту от генератора и создаёт реальные GameObject’ы через PrefabCatalog, расставляет врагов и предметы. 5.3 Добавить интерактивные объекты: двери (требуют ключ-карту), ящики (можно разбить для поиска предметов), компьютеры (активируют мосты/открывают двери). ● Случайно расставлять врагов и предметы в комнатах. 5.4 Создать LevelManager – управляет списком сцен, переходами между ними и передачей состояния игрока ● Используй сериализацию/передачу данных (например, сохранить текущее здоровье) при смене сцены. ● Добавить триггеры LevelTrigger на уровне (например, LevelExit), при входе в которые вызывается событие ● Например, когда игрок доходит до конца уровня — грузится новая сцена. ● На момент перехода показывать модальное окно с названием уровня 5.5 Реализовать несколько карт (локаций), между которыми игрок может переходить (DeveloperLevel, LevelOne, LevelTwo, etc.) ● Уровни можно загружать из отдельных файлов или префабов — не надо делать всё в одном мире. 5.6 Сформировать документацию для GDD по генерации карт 6. Сохранение/загрузка прогресса (метапрогрессия): Создать систему сохранения в файл (JSON или бинарный) данных о разблокированных предметах, максимальном пройденном уровне и т.д. 7. Readme-файл в формате Markdown, где: - Кратко описан проект (что за игра, особенности). - Указано, как ее запустить. - Есть скриншоты (по возможности).	12.05.2026	15.05.2026	
		17.05.2026	
		18.05.2026	
		17.05.2026	
		17.05.2026	
		17.05.2026	
		17.05.2026	branch Phase2.5-ProcedurallyWorldGeneration
		17.05.2026	
		17.05.2026	
		17.05.2026	
📅 Фаза 3: Углубление геймплея и атмосферы (3-4 недели)		23.05.2026	
Цель: добавить механики, приближающие игру к Dino Crisis.			
Задачи:			
1. Инвентарь с сеткой и комбинированием: Реализовать UI инвентаря в виде сетки ячеек (как в RE - Spatial Inventory). Предметы должны занимать определенное количество ячеек в инвентаре Возможность перетаскивать и вращать предметы, комбинировать (например, порох + гильза = патрон). Ограничение по весу предметов и количеству ячеек. Предметы должны иметь набор параметров - количество в единице, вес, категория (расходник и т.п.)		17.05.2026	
2. Улучшенное взаимодействие с окружением: Введение класса Actor в движок, рефакторинг классов Player и Enemy под наследование от Actor Добавить интерактивные объекты: двери (требуют ключ-карту), ящики (можно разбить для поиска предметов), компьютеры (активируют мосты/открывают двери). Компонент InteractableComponent , который при нажатии клавиши действия вызывает событие.		19.05.2026	
3. Звуковое оформление: Добавить звуки шагов, выстрелов, рычания динозавров, открывания дверей. Использовать AudioComponent с пространственным звуком (панорамирование в зависимости от позиции).		20.05.2026	
4. Разнообразие врагов: Добавить несколько типов динозавров: (раптор — быстрый ближний бой, птеранодон — летающий, стреляющий, брахиозавр — медленный с большим уроном). Использовать разные спрайты и анимации через SpriteMovementAnimationComponent или новую систему анимаций на основе состояний.		22.05.2026	
5. Система анимаций: Создать AnimatorComponent , управляющий сменой кадров в зависимости от состояния (Idle, Walk, Attack, Hurt) Сейчас есть только SpriteMovementAnimationComponent на основе движения, нужно расширить.		24.05.2026	
6. Эффекты и обратная связь: Экранные эффекты (красный flash при получении урона). Частицы (кровь, искры) — можно реализовать через простую систему частиц.			